

CS4205 - Evolutionary Algorithms

Assignment 2

This document describes the final practical assignment of the TU Delft course Evolutionary Algorithms (CS4205). This assignment will count for 30% of the final grade and can be done on one of the different topics listed below, each with a respective supervisor:

- *Evolutionary Van Gogh* (Single-Objective Discrete Optimization)
Supervisor: Koen van der Blom (koen.van.der.blom@cw.nl)
- *Evolutionary Van Gogh* (Multi-Objective Discrete Optimization)
Supervisor: Johannes Koch (johannes.koch@cw.nl)
- *Evolving a Lunar Lander with Genetic Programming*
Supervisor: Gloria Pietropolli (gloria.pietropolli@cw.nl)
- *Circles in a Square* (Single-Objective Real-Valued Optimization)
Supervisor: Peter Bosman (peter.bosman@cw.nl)

A more detailed description for each of these assignments is provided in the appendix.

The basic principles of the assignment are the same across the topics. This assignment is expected to be done in groups of 5 students. You are supposed to self-arrange into groups, there is a discussion forum for group finding on Brightspace to help you with this. Groups of less than 5 students may receive additional members or be split up at the discretion of the teaching staff.

Once you have a group, each of you needs to sign up on the same group in the Brightspace group interface. We will assign the topics to groups using a preference-maximization approach. For this, prior to the start of this assignment, each group of students must denote their relative preference for each of the 4 possible exercises in the Brightspace survey. The deadline for signing up for a group *and* for indicating your topic preference is **Sunday, May 10, 2026, 23:59**.

Process

The document for each topic contains a section with a recommended timeline. This timeline is purely a suggestion to make sure you are on track with your progress and will be able to deliver on time.

Weekly progress meetings between the group and the designated supervisor are offered if the group desires. During each of these meetings, you are requested to give an update on your progress and you are given the opportunity to ask questions. For the scheduling of these meetings, each group will be individually contacted by their designated supervisor.

Deliverables

Each group must hand in the following deliverables on Brightspace in a single zip file prior to the deadline of **Sunday, June 7, 2026, 23:59 CET**.

- All source code used to produce the final experimental results, as well as the final results themselves in a zip file.
- Presentation slides in PDF or PPTX format. The uploaded slides must be presented as-is. No changes to the slides are allowed after the deadline, and no additional material (animations etc.) besides the uploaded slides may be presented.

Presentation

You are required to present your work in a **9** minute final presentation on **June 10** or **June 11, 2026**. The schedule for the final presentations will follow at a later date.

Grading

A **complete** submission of the deliverables is required and the source code should run (else, it is a fail). The final grade is based on the following criteria, for which a rubric will be made available at a later stage:

- [60%] Content
- [20%] Presentation
- [20%] Defense

Evolutionary Van Gogh (Single-objective)

Topic: Single-objective Discrete Optimization



Figure 1: Vincent van Gogh’s “Wheat Field with Cypresses” (1889). *Google Art Project*.



Figure 2: One run of a Genetic Algorithm imitating Figure 1 with Voronoi cells. Shown are the best imitations at various generations.

A Problem Definition

Genetic Algorithms (GA) excel at solving many optimization problems. In this assignment we will be considering a rather unusual optimization problem: imitating art. The optimization problem in this assignment is to find an approximation of an existing painting, given a (limited) set of primitives. In order to “draw” shapes and therefore mimic an existing painting, we will be using Voronoi diagrams. To make a Voronoi diagram, we start by placing a number of points in an image space, where each point has a color associated with it. We then color the space as follows: For each pixel in the image, we give it the color of the closest point (by Euclidean distance). This partitions the space into regions (also called *cells*) around each point having the associated color of that point.

With this drawing strategy, we let the GA evolve imitations of the painting, using the conventional mechanisms of a GA (mutation, recombination, selection). A particular Voronoi diagram instance is encoded in an individual’s genotype as follows: Per point, a string of 5 integers (x, y, r, g, b) is stored describing the point’s position $((x, y)$, ranging from 0 to the image’s width or height -1) and color $((r, g, b)$, ranging from 0 to 255). These point information strings are concatenated to form a long string with all the points of the diagram, forming the genotype.

For the fitness of an individual, we need to establish how well it imitates the reference painting. We use `imgcompare`¹ for this, which subtracts the corresponding pixels in the imitation and the reference

¹<https://pypi.org/project/imgcompare/>

image from each other, and then assigns a score to this difference image (lower is better). Given this fitness, we use P+O tournament selection with size 4 to form the next generation.

B Goal of the Assignment

In this assignment, we want you to experiment with Genetic Algorithms through a hands-on example. You will start out with an existing implementation of a GA imitating Van Gogh’s “Wheat Field with Cypresses” oil painting (Figure 1). Note that we are not looking for a different way of drawing. We want to make improvements to the underlying components of the GA that lead to measurable improvement in the GA’s performance. A good first step that can be considered here is the current Black-Box Optimization (BBO) approach, and whether elements of Gray-Box Optimization (GBO) can be used.

C Materials

You are provided with a working version of the algorithm, using *one-point* crossover and a *random* initialization of points across image space. We want you to extend this implementation as described below. For this, you can use the Jupyter Notebook located in the provided ZIP file, which includes code to plot the progress of optimization over time and to display intermediate images. Please note that the installation was tested with python version 3.9.5 (other versions may need updates to the requirements).

It’s recommended to execute all experiments with a point budget of either **50** or **100** points and a generation budget of **500**. A run can take 3-5 minutes, so take this into account when planning your experiments. However, feel free to explore other experimental configurations if you think it makes sense. You can experiment with available hyperparameters, such as different population sizes and mutation strength.

D Recommended Project Timeline

This section describes an outline of the expected progress over the course of the assignment. Nevertheless, these guidelines are not enforced and only your final results are graded.

- Week 1.** Before the first meeting, get the provided code installed and running on your machines and make yourself familiar with the code. Decide on a preliminary experimental setup and baseline algorithm configuration(s), and produce graphs or tables with the results. Finally, write down ideas for improving the performance of the GA.
- Week 2.** Start with implementing the proposed changes in order to improve the performance of the GA on the image reconstruction problem. Formulate a full set of (at least) 3 hypothesis for why your improvements should work and obtain preliminary results on whether your changes are working as expected. Make sure at least 2 of these hypotheses consider new features you have developed.
- Week 3.** Refine your ideas based on the first experiments and set up final experiments to benchmark your new GA variant(s). Formulate a test for which you can motivate why it would confirm or reject your hypothesis and make sure your experiments are aligned with this test. We expect you to not only tell us what you observe but to also *explain* why you think you observe this.
- Week 4.** At the end of the last week you are required to hand in the final deliverables (code and presentation slides). Thus, you should finish the benchmarks for the algorithm and complete the presentation. The choice of the proposed changes must be motivated and the outcome of the comparison clearly conveyed. Make sure that the comparison is done in a fair and reproducible manner and that statistical significance testing² has been performed whenever meaningful to determine if the improvement (or deterioration) is statistically significant.

²Possibly using <https://github.com/sherbold/autorank> and/or <https://github.com/google-research/rliable>

Related Work

1. Darrell Whitley. A genetic algorithm tutorial (up to Chapter 4). *Statistics and computing*, 4(2):65–85, 1994. (*Also part of the mandatory reading material.*)
2. Sebastian Proost. Minimalist Art Using a Genetic Algorithm. 2020.
<http://blog.4dcu.be/programming/2020/02/10/Genetic-Art-Algorithm-2.html>

Evolutionary Van Gogh (Multi-objective)

Topic: Multi-objective Discrete Optimization



Figure 3: Vincent van Gogh’s “Wheat Field with Cypresses” (1889). *Google Art Project*.

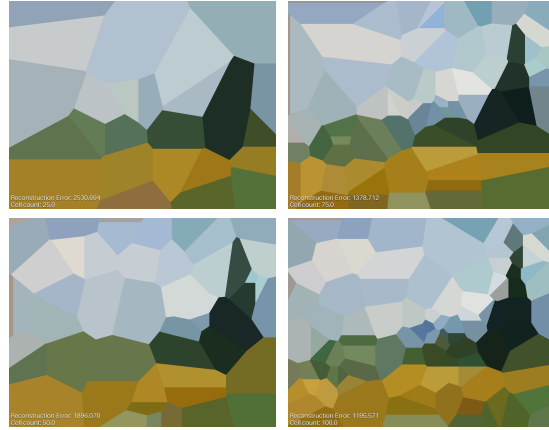


Figure 4: With increased Voronoi cell count, more accurate reconstructions are possible.

A Problem Definition

Genetic Algorithms (GA) excel at solving many optimization problems. In this assignment we will be considering a rather unusual optimization problem: imitating art. The optimization problem in this assignment is to find an approximation of an existing painting, given a (limited) set of primitives. In order to “draw” shapes and therefore mimic an existing painting, we will be using Voronoi diagrams. To make a Voronoi diagram, we start by placing a number of points in an image space, where each point has a color associated with it. We then color the space as follows: For each pixel in the image, we give it the color of the closest point (by Euclidean distance). This partitions the space into regions (also called *cells*) around each point having the associated color of that point. Additionally, the Voronoi cells can either be enabled or disabled, allowing reconstructions with varying Voronoi cell counts.

With this drawing strategy, we let the GA evolve imitations of the painting, using the conventional mechanisms of a GA (mutation, recombination, selection). A particular Voronoi diagram instance is encoded in an individual’s genotype as follows: Per point, a string of 6 integers (a, x, y, r, g, b) is stored describing if the point is active ($a \in [0, 1]$, where 1 enables the point and 0 disables the point), the point’s position $((x, y)$, ranging from 0 to the image’s width or height -1) and color $((r, g, b)$, ranging from 0 to 255). These point information strings are concatenated to form a long string with all the points of the diagram, forming the genotype.

To determine the fitness of an individual we consider two minimization objectives, the reconstruction error and the number of cells used. This allows trading off imitation accuracy and complexity. The reconstruction error used is the mean squared error (MSE), i.e. the squared difference between the target image and reconstruction per color channel, averaged over all pixels.

B Goal of the Assignment

In this assignment, we want you to experiment with multi-objective evolutionary optimization through a hands-on example. You will start out with an existing implementation of a GA imitating Van Gogh’s

“Wheat Field with Cypresses” oil painting (Figure 3). Note that we are not looking for a different way of drawing. Beyond making use of knowing how the genotype represents images you should *not* utilize the target image directly, e.g. to derive the optimal cell color. Compared to the single-objective version of this assignment, your primary goal is to improve the GA’s *multi-objective* performance to find better approximation fronts (Figure 4) instead of improving how the GA learns to imitate images. These goals can be closely related, so if you want to change how the algorithm searches, you need to motivate why this is interesting from the perspective of multi-objective optimization.

C Materials

You are provided with an implementation of the simple GA as shown in the lecture, but optionally using a truncation selection based on fast non-dominated sorting similar to the multi-objective NSGA-II algorithm. The code uses C++ for the algorithm and fitness function implementation and Python to run the provided example.

It’s recommended to use a point budget of **10-100** points and a generation budget of at least **500** (or similar in case of time/evaluation based comparisons). Furthermore, you are expected to run **30** repetitions per configuration, so take this into account when planning your experiments. However, feel free to explore other experimental setups if you think it makes sense.

D Recommended Project Timeline

This section describes an outline of the expected progress over the course of the assignment. Nevertheless, these guidelines are not enforced and only your final results are graded.

- Week 1.** Before the first meeting, get the provided code installed and running on your machines and make yourself familiar with the code. Decide on a preliminary experiment setup and baseline algorithm configuration(s), and obtain your first results. Finally, come up with ideas for improving the multi-objective performance.
- Week 2.** Start with implementing the proposed changes in order to improve the performance of the EA on the multi-objective image reconstruction problem. Formulate a hypothesis for why your improvement should work and obtain preliminary results on whether your changes are working as expected.
- Week 3.** Refine your ideas based on first experiments and set up final experiments to benchmark your new EA variant. Formulate a test for which you can motivate why it would confirm or reject your hypothesis and make sure your experiments are aligned with this test.
- Week 4.** At the end of the last week you are required to hand in the final deliverables (code and presentation slides). Thus, you should finish the benchmarks for the algorithm and complete the presentation. The choice of the proposed changes must be motivated and the outcome of the comparison clearly conveyed. Make sure that the comparison is done in a fair and reproducible manner and that statistical significance testing¹ has been performed whenever meaningful to determine if the improvement (or deterioration) is statistically significant.

References

1. Darrell Whitley. A genetic algorithm tutorial (up to Chapter 4). *Statistics and computing*, 4(2):65–85, 1994. (*Also part of the mandatory reading material.*)
2. Sebastian Proost. Minimalist Art Using a Genetic Algorithm. 2020.
<http://blog.4dcu.be/programming/2020/02/10/Genetic-Art-Algorithm-2.html>
3. Deb, K. and Pratap, A. and Agarwal, S. and Meyarivan, T. A fast and elitist multiobjective genetic algorithm: NSGA-II
<https://doi.org/10.1109/4235.996017>

¹Possibly using <https://github.com/sherbold/autorank> and/or <https://github.com/google-research/rliable>

Evolving a Lunar Lander with Genetic Programming

Topic: Genetic Programming



Figure 5: A successful lunar landing

A Problem Definition

Reinforcement learning (RL) is the task of training an agent to perform a certain task in an environment. The RL problem considered in this assignment is the Lunar Lander environment of Gymnasium. RL problems can be approached using EAs.

B Goal of the Assignment

The goal of the assignment is to gain a better understanding of how Genetic Programming works, why it works that way, and how it can be used in another application than the one seen in the lecture (symbolic regression), namely, RL. Examples of aspects to consider for improving GP are: using a different representation, crafting new atomic functions, designing new fitness functions, optimising coefficients etc.

C Materials

Documentation of the Lunar Lander environment is available at the Gymnasium website (discrete version). You are provided with a working GP library written in Python at https://github.com/CWI-EvolutionaryIntelligence/lunar_lander. The action space consists of 4 discrete actions, 0: do nothing 1: fire left orientation engine, 2: fire main engine, 3: fire right orientation engine. A multi-tree is implemented in the library to produce 4 action values, but you are welcome to use other solutions that can generate multiple outputs. The multi-tree is differentiable and programmed using PyTorch. An RL loop is set up for you, but it is up to you where and how to use this loop. You are further provided with a `requirements.txt` file that lists the libraries needed to implement your solution, as well as a Python notebook `solution.ipynb` that contains some utility functions. Please note that, in order to successfully install all the requirements, you need a python version equal to 3.10 or LESS. **All** the code you produce to tackle this assignment should be written in this notebook (unless a strong motivation for doing otherwise is brought forward and the supervisor agrees).

D Recommended Project Timeline

Week 1

- Got the baseline agent running
- Run baseline experiments (at least a few runs)
- Come up with potential ideas on what you want to do. Already start thinking about why these ideas are interesting and how you can implement and test it. Note that as the course is on evolutionary algorithms, your ideas should include modifying the existing algorithm in some way

Week 2

- Implemented at least one new feature
- Set up experiments to test the performance of the baseline and the improved agent(s)
- Started working on the presentation

Week 3

- Run proper experiments (multiple runs)
- Tune your method
- If possible, add other improvements or variation
- Draft of the presentation ready

Week 4

- All experimental results
- Create plots
- Evaluated your implemented improvements - Do they work? Why? What evidence do you have?
- Finalized the presentation

E Deliverables

The completed assignment should be uploaded to Brightspace (see the general guidelines). Important: For this assignment, the following applies to the final deliverable:

- As mentioned before, all code should be contained in the notebook `solution.ipynb`.
- The notebook should also contain two (or more) GIFs: one showing the behaviour of the baseline agent (i.e., the agent obtained by the baseline GP) and one (or more) showing the behaviour of the improved agent(s) (depending on how many improvements you realise). See, e.g., <https://stackoverflow.com/questions/51527868/how-do-i-embed-a-gif-in-jupyter-notebook> for how to include GIFs in a notebook.

Related Work

1. Poli R., Langdon W.B., McPhee N.F., 2009: A field guide to genetic programming — Part II: Advanced genetic programming. (Available as reading material for lecture 3.)
2. <https://en.wikipedia.org/wiki/Reinforcementlearning>
3. Hein, D., Udluft, S. and Runkler, T.A., 2018. Interpretable policies for reinforcement learning by genetic programming. Engineering Applications of Artificial Intelligence, 76, pp.158-169.
4. Zhang, H., Zhou, A. and Lin, X., 2020. Interpretable policy derivation for reinforcement learning based on evolutionary feature synthesis. Complex & Intelligent Systems, 6(3), pp.741-753.

Circles in a Square

Topic: Single-Objective Real-Valued Optimization

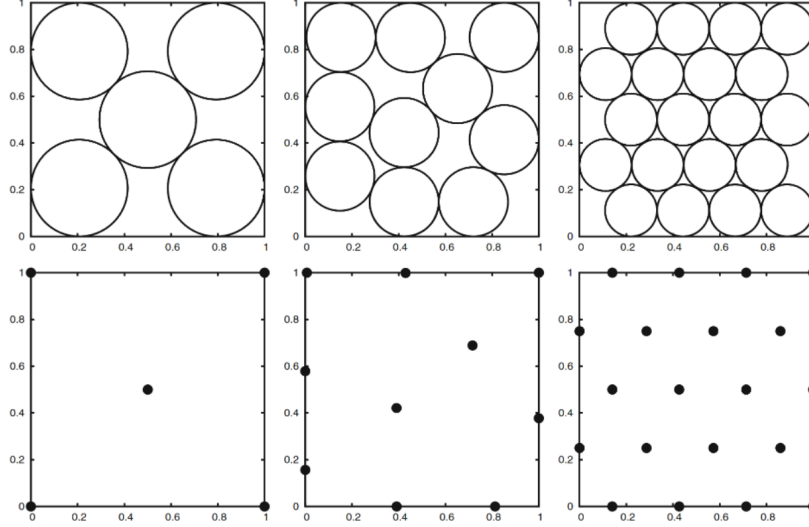


Figure 6: Top row: optimal circle packings for 5, 10, and 20 circles. Bottom row: corresponding point scatterings [1].

A Problem definition

This final assignment considers a scalable and real-valued optimization problem called "Packing n Circles in a Square (CiaS, see, e.g., [1])." This problem has many applications in real-world optimization problems like loading containers, communication networks, facility dispersion/layout, etc. Even though CiaS can be compactly formulated and efficiently evaluated, it is not trivial to solve and has interesting characteristics that differ from many other commonly used benchmark problems. For at least 2 circles, the CiaS problem is equivalent to finding an optimal scattering of points in the unit square. To obtain the optimal scattering of points, the smallest distance between any pair of points has to be maximized. Examples of optimal circle packings and their corresponding point scatterings are shown in Figure 6. The fitness function for the point scattering problem is as follows:

$$f_{scatter}(\mathbf{x}) = \min_{0 \leq i < j < \ell/2} \|(x_{2i}, x_{2i+1}) - (x_{2j}, x_{2j+1})\|$$

B Goal of assignment

To obtain state-of-the-art performance on CiaS is not trivial, as specific properties of the CiaS problem must be considered and exploited. It would be interesting to see to what extent EAs can be tuned to CiaS to also obtain state-of-the-art results with an EA. You will be doing so by improving one of the provided EAs and perform a performance comparison before and after it has been tuned for CiaS. Importantly, it should not be any of the improvements or variations tried in the published article in which AMaLGaM and CMA-ES were tested on this problem ([1]).

An especially useful resource is the Packomania website [2]. On this website, a large list of the optimal, or best known, packings is maintained. Currently, the list goes up to 9996 circles, with some intermittent omissions. You can also find other references and resources on the website.

C Provided materials

A base implementation of AMaLGaM with a full covariance matrix is provided in C. If you wish, you can choose other starting code from the web for a different real-valued evolutionary algorithm (such as differential evolution, particle swarm optimization, classic evolution strategies, or CMA-ES), but this is at your own discretion and begets no implementation support, should this be needed. Importantly: stick with **one** EA and try different improvements for that EA, the assignment is explicitly **not** about trying different EAs and comparing them.

D Recommended timeline

This section describes an outline of the expected progress over the course of the assignment. Nevertheless, these guidelines are not enforced and only your final results are graded.

- Week 1.** Your assignment is to improve the baseline performance of an EA and report the findings. To measure the baseline, implement the fitness function, benchmark the standard EA, and produce graphs or tables with the results. Make yourself familiar with the code and think of ideas for the improvement that you are going to make (see week 2).
- Week 2.** Start with implementing the proposed changes in order to improve the performance of the EA on the CiaS problem and achieve some preliminary experimental results.
- Week 3.** Refine your ideas based on first experiments and set up final experiments to benchmark your new EA variant. Formulate a hypothesis for why your improvement should work as well as test for which you can motivate why it would confirm or reject your hypothesis. Make sure your experiments are aligned with this test and perform all experiments.
- Week 4.** At the end of the last week you are required to hand in the final deliverables (code and presentation slides). Thus, you should finish any experiments still pending and complete the presentation. The choice of the proposed changes must be motivated and the outcome of the comparison clearly conveyed. Make sure that comparisons are done in a fair and reproducible manner and that statistical significance testing has been performed whenever meaningful to determine whether any observed improvement (or deterioration) is statistically significant.

References

- [1] Peter A.N. Bosman and Marcus Gallagher. The importance of implementation details and parameter settings in black-box optimization: A case study on gaussian estimation-of-distribution algorithms and circles-in-a-square packing problems. *Soft Comput.*, 22(4):1209–1223, feb 2018.
- [2] Eckard Specht. Packomania website, Feb 2022. www.packomania.com (accessed: 30.03.2022).